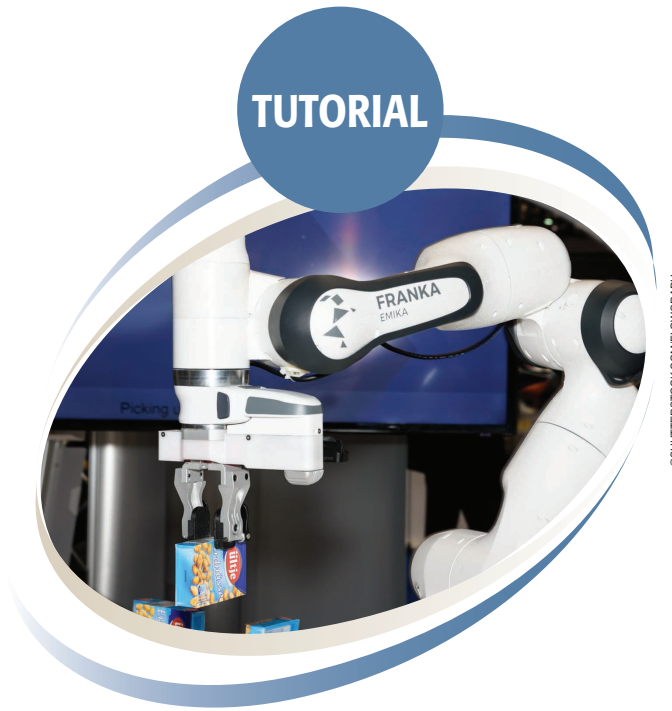


TUTORIAL



The Franka Emika Robot

A Reference Platform for Robotics Research and Education

By Sami Haddadin, Sven Parusel, Lars Johannsmeier, Saskia Golz,
Simon Gabl, Florian Walch, Mohamadreza Sabaghian,
Christoph Jähne, Lukas Hausperger, and Simon Haddadin

The importance of robots for industry, research, education, and society as a whole is steadily increasing as reflected by the number of available systems and installed robots, not only in industry but also in the public sector and households.

Software-only robotics researchers usually rely on commercially available robots which, in the case of manipulators, are primarily designed for industrial purposes and are often far from their needs. This article is a hands-on tutorial on the Franka Emika robot, the first series of industrial artificial intelligence (AI)-ready tactile robot platforms. Beyond industrial use, the systems can be seamlessly expanded to fulfill the demands of research and

education across all robotics and AI disciplines. To satisfy the needs of such a wide variety of fields, it provides three different interfaces: Desk, a high-level app-based user interface for easy and fast task programming; Robot Integrated Development Environment (RIDE), a command-based programming environment used to create high-performance robot skills that enables programming custom apps and integrating external sensors; and the Franka control interface (FCI), a 1-kHz low-level torque and position control interface that exploits the also-available Lagrangian dynamics robot model. We take a close look at implementations with all interfaces, ranging from simple solutions, apps, and controllers to robot-learning examples illustrating how to exploit all the advantages of this platform in ongoing robotics research and education.

Digital Object Identifier 10.1109/MRA.2021.3138382

Date of current version: 25 February 2022

Introduction

Robotics research involves the development of both new hardware and software. With the increasing popularity of AI, software-based robotics research is currently booming. As a software-only branch of development, it must rely on readily available high-performance robotic platforms, which, in the case of manipulation, are mostly limited to industrial robotic arms. Today, manufacturing is the most relevant commercially viable use case for automated manipulation. It drives and shapes the development of new robot arms and consequently, the programming interfaces that these robots provide are strongly oriented toward constrained industrial solutions. To work with these robots, research labs must overtake many difficulties and invest a significant amount of work to first set them up and then reach the desired signals or high-level commands that their specific application domain requires. In addition, the diversity of applications that a research robot should cover is very broad. Robotics encompasses many fields of computer science and engineering, including signal processing, control, planning, computer vision, machine learning (ML), or neighboring fields such as cognitive and social neuroscience. The requirements of researchers in all these areas differ significantly. For instance, from a control engineer perspective, the ideal platform offers 1-kHz-level joint-torque interface that exploits an energy-consistent Lagrangian dynamics robot model; it delivers industrial grade performance, with position repeatability of less than 0.1 mm and has accurate force/torque (F/T) sensing capabilities at the end effector.

In contrast, psychologists studying human-robot interaction focus on human behavioral aspects and require an effective way to program higher-level robot behaviors, such as possibly compliant, goal-oriented robot motions. This obvious focus gap between disciplines is one of the most challenging issues for robotics research platforms. Despite some progress in recent years where some available manipulators have started including a research interface into their products (either high or very low level), these do not fulfill the needs of such an heterogeneous variety of fields. In fact, there is possibly no unique interface for the whole application spectrum of robotics research. It thus becomes apparent that robot manufacturers have not previously considered robotics research and education in an integral use case during the conception of their products at scale.

From a researcher's point of view, the influence of a specific platform and its impact on his/her work cannot be overstated. At first glance, its application programming interface (API) dictates the necessary implementation effort, while its hardware and low-level interface determines the robot-specific control performance of new approaches. However, the plurality and flexibility of interfaces strongly conditions the success of multidisciplinary works and joint projects. The level of adoption by the community and ease of reproducibility of competing methods also defines the quality of potential evaluations, objective comparisons and available open source code. For dissemination and outreach activities, ease of use and interactivity are key; but also, an overarching platform that is likewise designed for educational purposes boosts and deepens the dissemination of research results into the general public. With this holistic researcher-centric perspective in mind (see Figure 1) for the development of the Franka Emika robot, we identified the following key goals. Note that these goals are deliberately formulated from a rather high-level perspective and serve primarily as motivation for the presented platform.

- **Multidisciplinary accessibility:** Researchers should be able to easily access the signals, sensor measurements, and commands that their applications require. Control researchers require low-level access to the robot, e.g., direct joint-torque control over a sufficiently fast real-time connection (1 kHz), while psychologists do not require the programming of custom low-level controllers from scratch for their experiments. For them, a higher-level interface is significantly more effective.

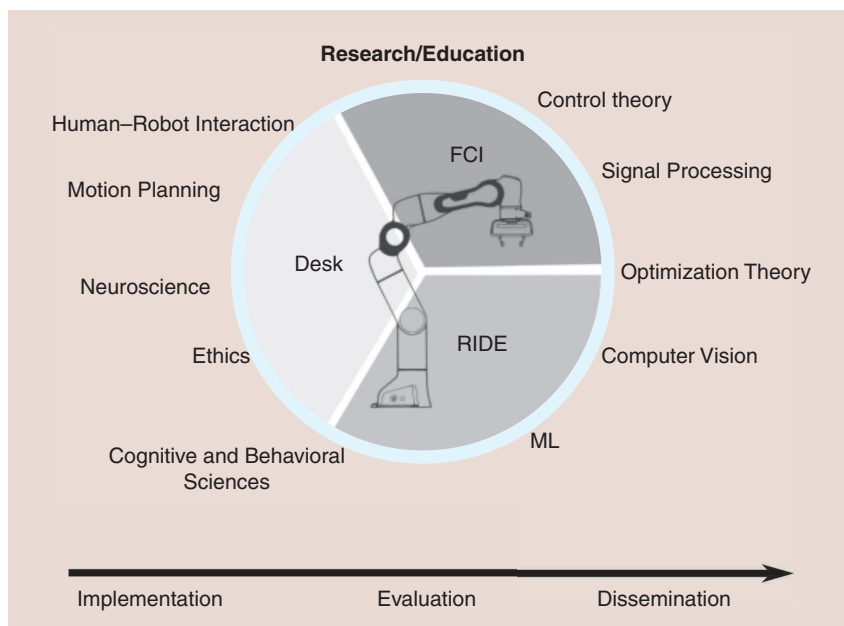


Figure 1. The holistic approach of the Franka Emika robot, with three different programming interfaces that satisfy the needs of most robotics research and education fields.

- *AI-ready*: Robot systems are the embodiment of AI. An AI-ready reference platform should provide easy access to all sensor data at high frequency, and fast control with minimal latency. It must also excel at interaction control to enable safe environment exploration, a key requirement for robot-learning approaches. It should also support the development of AI solutions with AI libraries provided in house for all robotics disciplines.
- *Interaction control*: A stable and sensitive physical interaction with the environment is the defining characteristic of manipulation. A robot that excels at controlling it, capable of accurately adapting its rendered compliance and regulating its applied force, is the only way to advance the state of the art.
- *Applications*: A reference platform for manipulation must be capable of performing the widest range of manipulation tasks. It must contain high-performance sensors, actuators, and control algorithms that enable all possible behaviors encompassed in robotics research.
- *Easy setup*: It requires some degree of expert knowledge to properly set up an industrial robotic platform. A reference platform for research, and especially for educational purposes, should be easily set up in a matter of minutes by every layperson.
- *Sharing and community*: A big fraction of the development time of any new method in robotics research is given by its software integration, especially for a new platform. An even bigger fraction consists of reproducing research from other researchers, which is unfortunately often not even possible. A robot for researchers should encourage the community to have an in-house curated platform for technical discussion and code sharing.
- *Target audience*: The target audience of traditional robotic systems is usually rather narrow and limited to industrial engineers or robotics researchers, hindering the impact of their work. A research platform should be as close as possible to the general public and connected with all phases of education; it should offer accessible interfaces and interaction modes for everyone to be the embodiment that prepares not only future engineers but also an open door into robotics research for society as a whole.
- *Affordability and availability*: The impact and potential reach of a platform is first affected by the quality of its features but mostly defined by its user base. Most industrial robots still require significant financial investments, which prevents them from being available to medium-sized companies, researchers, and educational institutions in large numbers. In fact, no robot platforms are currently mass produced, which affects both pricing and availability. A research platform will only succeed if it's beyond widely accepted; it must be affordable and available worldwide.

The remainder of this article is a hands-on tutorial on the Franka Emika robot, the first series of industrial, AI-ready tactile robot platforms built not only for industry

but also robotics research and education with the aforementioned goals in mind. To see the platform's scope on a technology level, the interested reader may refer to the official data sheet [1], which provides further details and references to the accordingly applied standards. Note that we refer to *AI-readiness* as the ability of the platform to 1) provide all sensory data (processed and raw) at the same 1-kHz real-time frequency used for internal control purposes. This includes measured joint data, such as joint/ Cartesian position; velocity as well as link-side joint-torque sensor signals; and an estimation of externally applied joint torques and wrenches as well as various further collision and contact information. Also, the forward kinematics of all robot links, Jacobian matrices of all robot joints, the inertia matrix, Coriolis, and centrifugal and gravity vectors are available to the maximum accuracy, 2) allow for low-level, desired joint-torque access at the same 1-kHz frequency used for internal purposes and with a very low latency, i.e., full customization with regard to physical manipulation, and 3) provide easy-to-access interfaces in common programming languages such as C++ and Python as well as connections to common research frameworks such as the Robotic Operating System (ROS) or MATLAB, which are widely used in robotics and AI research. The amount of proprioceptive low-level data that are available to feed AI-algorithms sums up to 2.3 MB/min.

All of the tutorials shown in this work are also available in video form see [8]. The source code from the examples is available at [9]. Note that we use a single example peg-in-hole task, namely, inserting a USB plug, throughout the article. For the readers who are already familiar with the basics of the platform and would like to, e.g., read about ML integration, they may skip ahead to the later sections.

Related Platforms

The suitability of a robotics research platform is first and foremost inherently tied to the technology powering it and the use cases it enables. The control paradigms, sensing technologies, low-level interfaces, and applications of the currently most widespread industrial manipulation platforms in research are summarized in Figure 2. On the joint level, robots that regulate only position or velocity, such as Universal Robot (UR) robots or ABB Yumi, neglect the dynamic characteristics of the joint. As a result, they operate with very stiff position control. To perform contact manipulation tasks such as insertion or polishing, they require an additional F/T sensor at the end effector, like e.g., the URe series. However, cost-effective F/T sensors typically provide low sensitivity and limit the range of applicable use cases, which remain mostly position/velocity based, such as pick and place or machine-tending tasks. In contrast, joint-torque-controlled robots [10]–[12] regulate the joint-side torque, which explicitly incorporates the dynamic properties of the robot into the control loop. Joint-torque sensing is implemented either

by measuring the current combined with a low gear ratio [13] (Barrett WAM Arm), by exploiting the implicit compliance of series elastic actuators [14] (Rethink Robotics' Baxter), or by adding joint torque sensors [10]–[12] (KUKA iiwa). This enables controlling the robot's impedance [15]–[20], i.e., its compliance as it interacts with the environment or with humans [12], [21], as well as the desired interaction force.

In addition to basic impedance control, the Franka Emika robot implements all impedance, reflex, and tactile control features [22], including compliant teaching, safety [23], [24], collision and contact detection, and collision and contact reflexes [25]. It relies on highly sensitive strain-gauge-based torque sensors that, in addition to industrial-grade position performance, provide closed-loop force control performance capable of applying regulated forces as low as <0.05 N, which outperforms all other robots in terms of interaction control and sensitivity. This supports a variety of manipulation tasks, comprising sensitive testing, robust fault detection, force-profile tracking, screwing, or drilling. In addition, the Franka Emika robot provides in-house support for ML applications. It includes learning methods and services that enable accessible AI solutions (see the second row in Figure 2). Specifically, we refer to the ability for ML researchers to use platform-tailored tools with high-accuracy and low-latency measurements that allow for a focus on the specificities of ML without the concern for unnecessary control loops or the unavailability of accurate standard dynamics/kinematics models. As we designed for only the most essential safety functionalities in this mode, the robot behaves according to Lagrangian

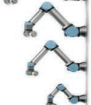
	 UR Robots  ABB Yumi  UR Re Series	 Rethink Robotics Sawyer  Barrett WAM  KUKA iiwa  Kinova Gen3	 Franka Emika Robot
Control			
Joint Level	Position/Velocity Control	Joint-Torque Control	Reflex and Joint-Torque Control
Coordination	N/A	Impedance Control	Reflex and Impedance Control
ML	N/A	N/A	Learning Basic Assembly Skills Manipulation Signatures
Force Sensing	Current + High Gear Ratio + Force-Torque Sensor	Series Elastic Actuators	Strain Gauges
Low-Level Interfaces	Position/Velocity + Force	Position/Velocity, Joint Torque, Force	Position/Velocity, Joint Torque, Force, Collision Behavior
Use Cases	Pick and Place, Machine Tending, Teaching With Pendant + Insertion, Polishing	Pick and Place, Machine Tending, Compliant Teaching, Compliant Insertions, Polishing, and Grinding	Sensitive Pick and Place, Sensitive Testing, Force Profile Tracking, Robust Fault Detection, Polishing, Grinding, Complex Sensitive Insertions, Screwing, Drilling, and Compliant Teaching

Figure 2. A technological overview of the most relevant currently available robot manipulators, their control paradigms, sensors, interfaces, and target use cases. The robot images are taken from [2]–[7]. N/A: not applicable; UR: Universal Robot.

dynamics (within the relevant bandwidth) so that the respective ML researcher can freely choose the approach to be followed. This makes it possible to set up ML-based applications with minimal configuration effort and tedious trial-and-error preparation. Note that despite it being regularly used in ML applications, we did not include the PR2 [26] in Figure 2 as it is no longer commercially available.

Concerning the APIs and interfaces that these robots offer for researchers, some of them stick to their industrial counterparts, while others offer explicit interfaces for research: for example, the KUKA iiwa provides a low-level, fast research interface. In contrast, the Franka Emika robot offers three strongly supported interfaces targeting three different levels of abstraction. It also allows for an easy interplay across interfaces, making multidisciplinary approaches and collaborations more accessible.

Programming With the Franka Emika Robot

The Franka Emika robot is an AI-ready tactile and affordable robot platform. The currently available Franka Emika robot is its 3-kg payload version. As depicted in Figure 3, it consists of the arm, a seven degree-of-freedom manipulator equipped with joint-side torque sensors in every joint, hand, two-finger gripper, and control, the processing unit that controls the robot. The arm is also equipped with the grip, which enables the guiding mode for kinesthetic teaching, and the pilot, a set of buttons that provides inputs to easily program the robot during teaching.

The Franka Emika robot provides researchers with the following three different programming interfaces depending on their research focus and the technical task requirements (see Figure 4):

- 1) Desk is a high-level programming interface for every layperson. It targets ease of use, intuitiveness, and programming speed. The Prototyping App Package includes several low-level apps such as compliance, force, or move to contact, which enable researchers to easily prototype and program standalone solutions.

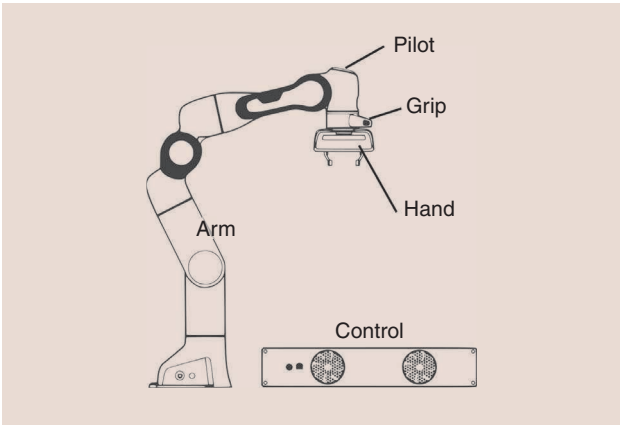


Figure 3. The components of the Franka Emika robot.

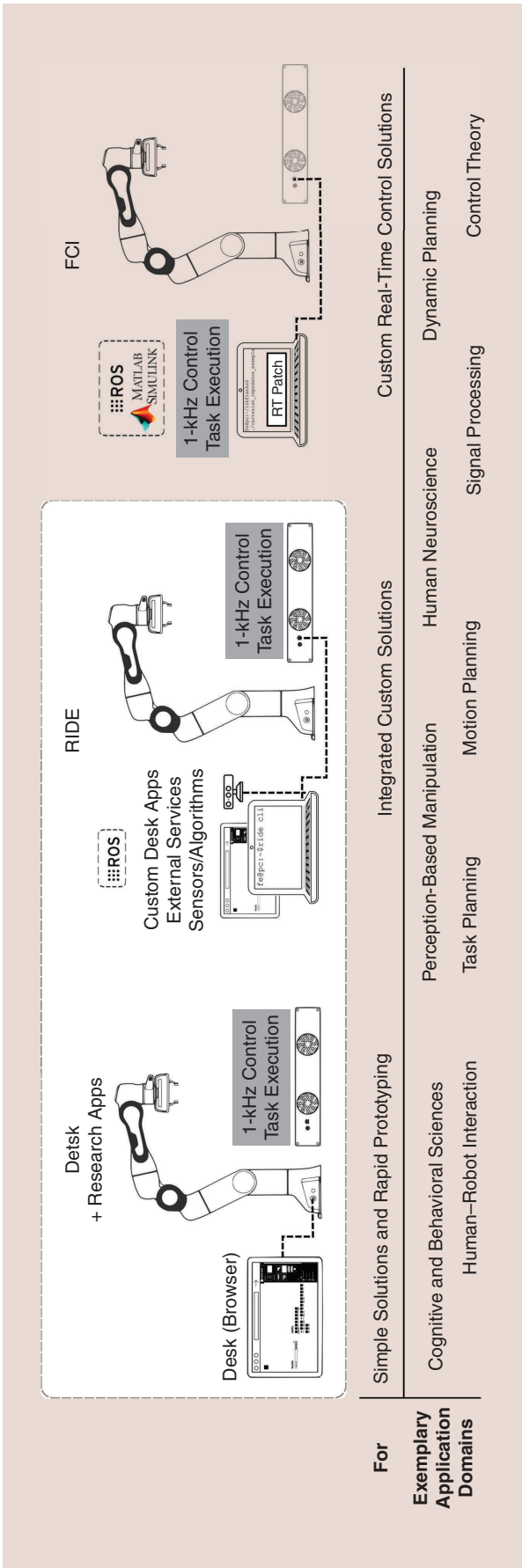


Figure 4. An overview of the three programming interfaces of the system, their hardware configurations, and target applications.

Desk, together with the Research Apps, is conceptualized to be the right interface for high-level programs, such as human-robot interaction studies or for rapid prototyping quick ideas.

- 2) The Robot Integrated Development Environment (RIDE) is the API for writing custom Desk apps and connecting external sensors, algorithms, and any other functionality to the system. It enables researchers to fully integrate their experimental setups into the Franka Emika system while at the same time exploit high-performance controllers of Control. The RIDE is the most suitable programming interface for research that does not require per-millisecond reactions, such as task and path-planning or perception-based manipulation.
- 3) The FCI enables researchers to bypass Control and run their own control algorithms in an external, real-time-capable PC at 1 kHz. It is the right interface to explore new low-level planning and control schemes.

The Desk and RIDE user and programming interfaces build on the Lingua Franka formalism. It was co-developed with the mechatronics and control system as a part of the Franka Emika robot. With this, the many interaction and reflex control capabilities as well as tactile commands/controls can be fully exploited at the user level and industrial technology readiness with its certification and safety requirements. Clearly, state-of-the-art robotic middlewares such as ROS, various Python libraries, or MATLAB have significantly lowered the entry barriers for researchers and professionals by providing interchangeable and reusable software components. However, in most real-world setups, there are still several distributed computers controlling a single robotic task, prone to laborious (re-)configuration and long setup times. Even today, there does not exist a commonly accepted solution to the intuitive and flexible programming of complex robotic systems with the ability to interact physically. Lingua Franka was thus conceptualized nearly a decade ago to fill this role and provide an industrial-grade programming formalism and environment. Analogous to the established standards in position-controlled industrial robotics, the complex requirements of collaborative and tactile robot capabilities were to be already matched during robot program development at the closest possible level to the control level. Instead of artificially adding these language layers later, ultimately leading to suboptimal solutions due to communication, delay, or real-time implications, the goal of Lingua Franka is to provide a high-performance, certifiable, and modular yet simple and flexible approach to the programming of modern tactile robots. To achieve this, Lingua Franka builds on reusable, hierarchical, easy-to-tune state machines, which can be combined to solve complex robotic tasks.

From a safety design point of view, the Franka Emika robot has multiple built-in hardware and software mechanisms, ensuring the safe and reliable execution of created programs. Each mechanism is activated under different

circumstances, bringing the system from the operational standard robot mode into a different safety-related state. Due to regulations, the robot can then be returned from any of these safety states only manually. Figure 5 depicts a schematic overview of the designed mechanisms:

- 1) *Emergency stop device*: The emergency stop device is integrated between the electrical power supply and controller for cutting off system power in case of emergency. This causes the robot to brake at maximum capacity and the locking bolts to mechanically lock all joints.
- 2) *External enabling device*: The external enabling device is plugged into the arm's base. When the external enabling device is half pressed and remains half pressed, it is possible to test and run automatic robot programs.
External activation device: The external activation device is plugged into the female X3 connector of the arm. Once the external activation device is in the position "up state," automatic execution of tasks can be carried out with the arm.
- 3) *Pilot*: The pilot is the user interface integrated directly into the arm for smooth pilot interaction with end effectors and Desk. On the grip of the pilot there are two gray buttons (guiding and enabling buttons), one on each side. By pressing them, the guiding mode is activated as long as the external activation device is pressed down, and the arm can be hand guided by the operator to show the Franka Emika robot's desired poses. Once the buttons are released, the arm can no longer be moved. In case these buttons are pressed while the robot is moving, it immediately stops and requires a manual reset via the external activation device.
- 4) *Self-collision avoidance*: The distance between all the robot shells (outer parts) is monitored at 1 kHz, and repulsive forces/joint torques are generated to avoid self-collision when the user moves the robot around in guiding/teaching mode. The repulsive forces help the user avoid collision

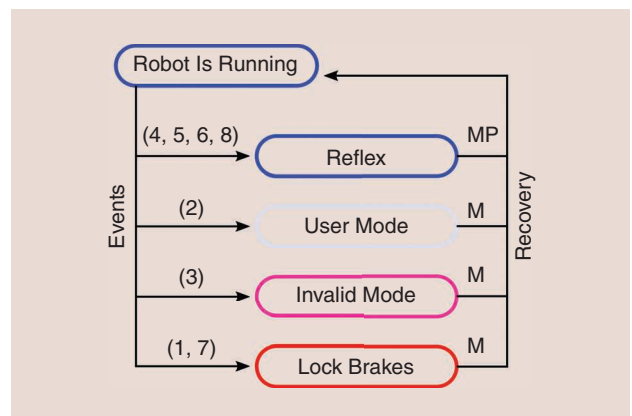


Figure 5. The various operating modes of the Franka Emika robot. The triggers that bring the system into the respective states are described in the text. The system can be recovered from a reflex programmatically (P) or manually (M). Any other state requires manual interaction to return to the initial operation mode. The colors around the states coincide with the colors of the base LEDs of the robot.

without stopping the robot or interrupting the task. During the automated/planned operation of the robot, collision avoidance has to be implemented by motion planning. The repulsive forces are deactivated for this mode to avoid unwanted changes to the user's desired trajectory. However, self-collision is still monitored, and in case of their hard violation, the robot stops moving and informs the user about the collision. Self-collision values are tuned based on robot parameters, such as maximum velocity and maximum torque/force, which can be generated in case of violation to protect the robot.

Virtual joint walls: Similar to self-collision avoidance, the joint limits of the robot are monitored at 1 kHz, and repulsive forces are generated to help users not violate the limits in guiding/teaching mode. In operation/autonomous mode, joint-position limits are still monitored. In case of hard violation, the robot stops moving and informs the user about the collision. Joint-wall values are tuned based on robot parameters such as maximum velocity and maximum torque/force, which can be generated in case of violation to protect the robot.

- 5) **Collision detection:** An F/T observer estimates the external F/Ts applied to the robot at 1 kHz. Depending on the tasks



Figure 6. The experimental setup for a *plug in* task consisting of the insertion of a USB plug into its corresponding socket. (Source: Franka Emika GmbH; used with permission.)

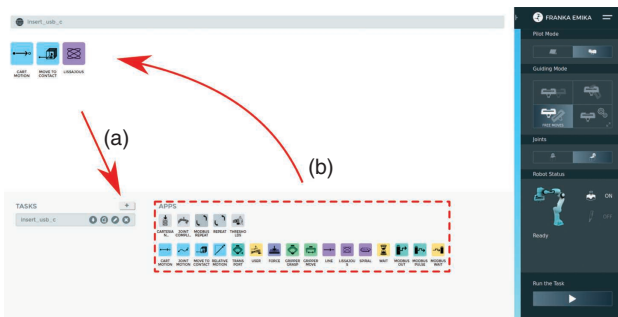


Figure 7. The programming phase of a *plug in* task with the prototyping app package. (a) The creation of a new task by clicking on the "+" of the task pane. (b) Dragging and dropping the apps from the app pane to the timeline. (Source: Franka Emika GmbH; used with permission.)

at hand, the user has the possibility to tune the maximum-allowable F/T thresholds and therefore the collision sensitivity of the robot. The threshold values are tunable, both via Desk and FCI.

- 6) **Soft system limits:** The robot stops if an internally defined limit for Cartesian or joint velocities or accelerations, joint torques or their derivatives, or power consumption is reached. These limitations are derived hardware-level system limits and ensure transiting into stop modes from which the robot can recover programmatically.
- 7) **Hard system limits:** The robot stops and locks its brakes if the internally defined hard limit for Cartesian or joint velocities or accelerations, joint torques or their derivatives, or power consumption is reached. These limitations are derived from the hardware-side system limits and are in place to avoid damaging the robot.
- 8) **Real-time communication:** The FCI is designed to give users the possibility to control the robot from their own computer. We documented a set of recommended minimum hardware requirements (for example, the recommended network card); however, if the user's PC cannot update the necessary control values at the specified 1-kHz control rate due to hardware/software problems, the maximally tolerated dropped packages are defined in libfranka. If this value is violated, an error triggers, informing the user about the unsatisfactory network quality.

In the following three sections, we program an illustrative plug-in solution (see Figure 6) with all three programming interfaces to showcase their strengths, limitations, and ideal use cases.

Programming With Desk

Desk is the web-based interface of the Franka Emika robot. It can run on the browser of any device, sharing the network with the robot. Desk offers an intuitive interface based on apps. Apps are high-level encapsulated robot programs that require only a few simple, context-aware parameters from the user side to work.

A task in Desk is programmed as a timeline, which represents a sequence of apps, depicted and executed from left to right. We use three apps in the tutorial, namely,

- The Cartesian motion app will position the robot in its initial pose.
- The move-to-contact app will move the robot toward the socket until it detects a contact.
- The Lissajous app will regulate a constant force in the z-axis and produces a wiggle motion in the (x, y)-plane for n seconds to insert the plug.

We also provide only the move-to-contact and Lissajous apps in the prototyping package [27]. The Cartesian motion app is not included in the package; however, it is by default installed on every robot.

Programming a task in Desk is easily done in the following two phases:

- 1) The task programming phase (Figure 7), which happens on the browser, where we drag and drop the three aforementioned apps from the “App” pane to the timeline.
- 2) The parameterization phase (Figure 8), which is conducted by physically interacting with the robot to select a pose or a set of poses by means of kinesthetic teaching, and using the pilot to navigate through the context menus of each app.

This simple process is done in a matter of minutes and requires no expert knowledge. The obvious ease of use and minimal programming time makes Desk the most suitable interface for rapid prototyping, simple human–robot interaction studies, and demonstrators.

Integration With Other Ecosystems

Desk is a high-level interface suitable for nonexperts and does not offer any specific support for other ecosystems on that level. Any custom integration into the system is to be done using RIDE. However, note that any timeline in Desk can also be programmatically triggered using the ride-cli tool (see the “Programming With RIDE” section for more details on the system architecture).

Programming With RIDE

Although the Prototyping App Package and Desk provide the fastest interface to program simple standalone solutions, when applications require custom features or integration with other sensors or systems such as a camera, RIDE offers researchers a way to program their own apps and services. In the following sections, we give a brief overview of the operating system running on Franka Emika robots, followed by a tutorial on how to write a custom app and service.

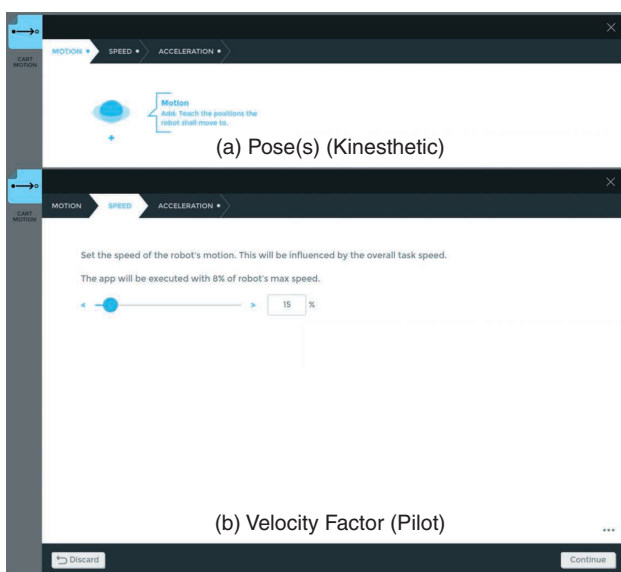


Figure 8. The exemplary parameterization menus for a pose (kinesthetic teaching) and a velocity factor (pilot). (Source: Franka Emika GmbH; used with permission.)

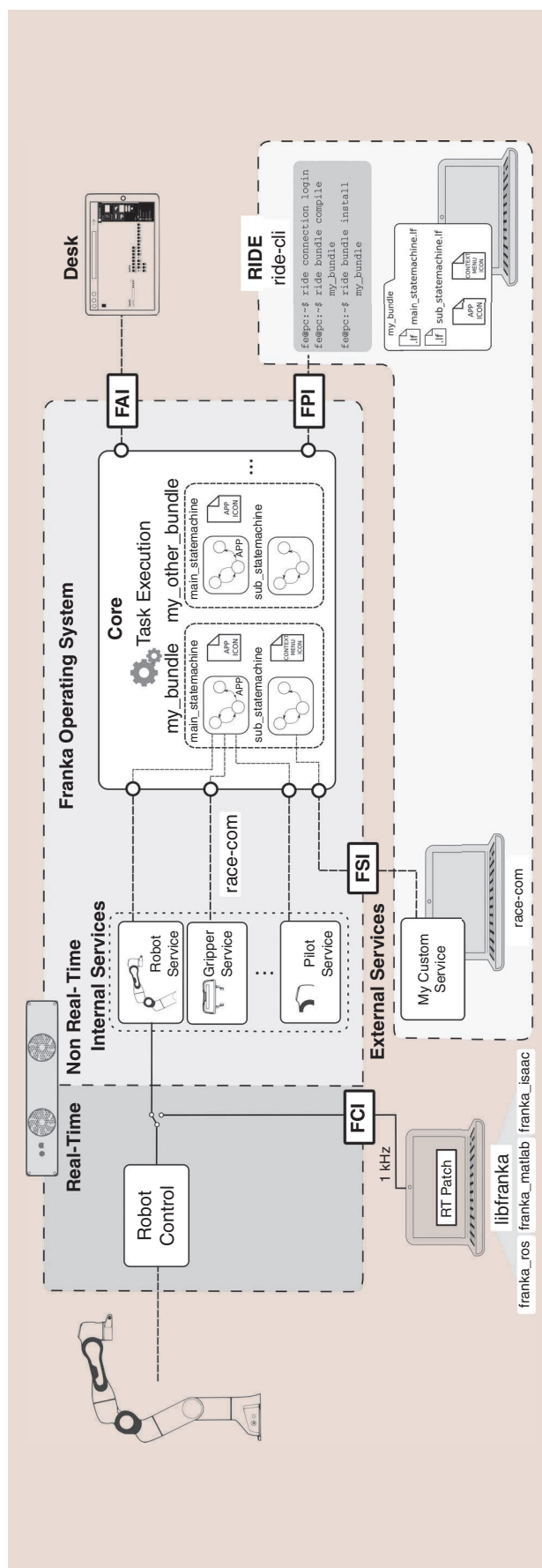


Figure 9. An overview of the system architecture for Desk, RIDE, and FCI. FAI: Franka Application Interface, FPI: Franka Professional Interface.

System Architecture

The Franka Operating System architecture, shown in Figure 9, is composed of the following main components:

- State machines are the basic units of execution in the system. They call operations from services, execute scripts, and may contain other state machines. All apps as seen in Desk are state machines. They are grouped together into bundles, which also store-related resources such as icons. State machines are programmed in a domain-specific language called *Lingua Franka* and stored in .lf files.
- Services are nodes that run on Control (internal services) or any external computer (external services). They communicate with the core via the race-com interface and can provide operations and publish events periodically. The most relevant service is the robot service, which provides all the operations to command the robot, such as “move” or “applyForce,” and continuously publishes events containing the current robot state, which is given by the joint position, velocity, end-effector pose, or external force among other signals. Other internal services are hand, pilot, or modbus (a standard communication protocol for industrial devices). Users may also implement their own services to make external signals, sensors, and algorithms accessible to state machines.
- The Core orchestrates the execution of state machines, stores installed bundles, and ensures that newly uploaded bundles are consistent with existing ones. It also implements the following three system interfaces:
 - 1) The Franka Application Interface, which is used by Desk to command and monitor the system.
 - 2) The Franka Professional Interface, which is used by ride-cli to manipulate, run, and install state machines in the system.
 - 3) The Franka Service Interface (FSI), which is used by the race-com library to connect external services to the system.
- Clients represents the access point for users. Currently, there are the following two clients in the system:
 - 1) Desk, as shown in the previous section, can start the execution of timelines (which are sequences of state machines) and trigger other specific functionalities, such as opening the brakes or turning the power on or off.
 - 2) ride-cli, the main development tool, provides a command-line interface to start, compile, install, monitor, and debug the execution of state machines.

To develop custom apps and state machines, you will need the race-com libraries required for writing external services and also the ride-cli tool to install, parse, and compile state machines and context menus for your apps. As an example, in the following, we first implement a plug-in state machine that encapsulates the behavior we presented in the previous section into a state machine that adds additional feedback and monitoring. We then program a service that provides the socket pose and adapt the app accordingly.

Plug-In State Machine

To write our own state machine, we first create our own bundle. The RIDE tool provides a simple shell command that builds the necessary folder structure. The command

```
ride bundle new plug_in
```

creates a plug_in folder with a manifest.json file that defines the bundle; a resources folder, which typically stores the icon files; and a sources folder, where you find a plug_in.lf file with an empty state machine. To program the desired plug-in behavior in Lingua Franka, you can download the code from the tutorial repository. Here we briefly explain the most relevant parts of the main state machine:

```
plug_in {  
  -- This is a comment.  
  -- Port definitions  
  port success child("monitor_pose")  
    .port("is_inserted")  
  port error child("move_to_socket_pose")  
    .port("error") or  
    -- ...  
  
  -- Parameter definition  
  parameterType {  
    -- ...  
  }  
  
  resultType {  
    string error_cause;  
  }  
  
  --> move_to_socket_pose <-  
    move_via_with_move_configuration {  
      port success -> set_move_configuration  
    } where parameter.socket_motion  
  
  -- set collisions thresholds  
  set_move_configuration <-  
  set_move_configuration {  
    port success -> insert  
  } where parameter.insertion_move_  
    configuration  
  
  -- Concurrent execution  
  barrier insert {  
    -> wiggle  
    -> press  
    -> monitor_pose  
  }  
  wiggle <- move_velocity {  
  } where parameter.wiggle_motion
```

```

press <- apply_force {
} where parameter.force

monitor_pose {
  port is_inserted result~=nil and result

  parameterType{
    [16]float target_pose;
    [3]float tolerance;
  }

  resultType bool

  action service("robot").event
  ("sensor_data")~=nil
  @{
    local current =
      service("robot")
        .event("sensor_data").O_T_EE.
        pose
    local x =
      abs (parameter.target_pose[13]
        - current [13])
    local y =
      abs (parameter.target_pose[14]
        - current[14])
    local z =
      abs (parameter.target_pose[15]
        - current[15])
    if x < parameter.tolerance[1] and
      y < parameter.tolerance[2] and
      z < parameter.tolerance[3] then
      setResult (true)
    end
  }@

} where {
  target_pose: parameter.hole_pose;
  tolerance: parameter.tolerance;
}

-- In case of any error of any state, set here
-- the error_cause defined in resultType, e.g.:

-- ...

action child("set_move_configuration")
  .port("error") @{
    setResult({error_cause =
      child("set_move_configuration")
        .result.error_cause})
  }@

-- ...
}

```

Each state or state machine is defined as `my_state {}`. At the beginning of the file, we define a state `plug_in`. Inside a state, we can define an arbitrary amount of child states. The initial one is defined with the operator `-->`, so our `plug_in` app will first start with the `move_to_socket_pose` state, which will move the robot to the socket pose. The states can have ports, which make the execution transition to other states. They are defined with the reserved `port` keyword. For instance, our `set_move_configuration` state will transition to the `insert` state once it triggers the success port. The states may also import other state machines already installed in the system. For instance, the `move_to_socket_pose` state is importing the `move_via_with_move_configuration` state machine with the `<-operator`. To check the installed state machines that you can reuse in your system, you can use the

```
ride node list
```

command. The Lingua Franca language also allows for concurrent execution of states: with the reserved `barrier` keyword, you can transition to several states at the same time. In our example, the `insert` state is starting the concurrent execution of the `wiggle`, `apply_force`, and `monitor_pose` states. This will allow the robot to wiggle in the (x, y) -plane while, simultaneously, apply and regulate a constant force along the z -axis and monitor the robot pose. The states may also have actions, denoted with the `action` keyword, that will run every time the condition specified is fulfilled. For instance, the action in the `monitor_pose` state will trigger its script, which evaluates whether the robot pose reached the target pose, every time the robot service delivers sensor data.

The bundle can be compiled and uploaded to Control using the Makefile included in the tutorial repository. To run the state machine, you can either run it with Desk by embedding it into an app as described in the repository, or you can simply use the `ride-cli` tool to start any installed state machine with

```
ride cli start <state-machine>\
' {param1: value1; param2: value2; ...}'
```

Vision-Based Socket-Detection Service

The state machine described in the previous section is capable of performing an insertion for any given pose. Let us now assume that a computer vision module is recognizing and identifying sockets in the scene, and we would like to provide its pose to the state machine. To this end, we will program an external service that implements an operation `getSocketPose` and periodically publishes an event `socket_info` with the number of sockets detected. To implement a service in C/C++, we first define the operation type in a `.op` file. All the operations either succeed or fail and must have at least a request, result, and an error type. As a minimal example, the `GetSocketPose.op` file contains

```
{ } request;
{
    [16]float socket_pose;
} result;
string error;
```

which defines an empty operation call that returns a 16-dimensional float array representing the homogeneous transformation of the socket in the robot base frame in case of success, and a string in case of error. Similarly, event types are defined in .ev files. The SocketsInfo.ev contains

```
{int number_of_sockets};
```

which defines a message given by an integer.

With operations and event types defined, the race-com libraries will automatically generate the necessary headers for our CMake project. The service is implemented in C/C++ in the socket_service.cpp file. The file first includes the necessary race-com headers as well as the auto-generated headers for the GetSocketPose operation and the SocketsInfo event.

```
#include "messages/GetSocketPose.h"
#include "messages/SocketsInfo.h"
#include <racecom/racecom.h>
// ...

void getSocketPoseCb(CallHandle h,
                    // call handle
    struct GetSocketPose_request* request,
    void* hint) { // user data
    // pose: column major
    std::array<double, 16> socket_pose;

    // ...
    // Fill in socket_pose with your
    // computer vision solution
    // ...

    // Prepare result
    // First allocate array for pose
    auto* socket_pose_c_array =
        static_cast<double*> (
            allocateArray(16, sizeof(double)));
    // ... then copy content from pose
    // to the allocated C-array
    std::copy(socket_pose.begin(), socket
        _pose.end(), socket_pose_c_array);
    GetSocketPose_result result = {
        socket_pose_c_array};

    // Return result
    succeedOperation(h, &result);
    free_GetSocketPose_request(req);
}
```

The getSocketPoseCb callback is defined as a C-function that receives the handle, request, and any necessary additional user data. Once the operation is computed, it triggers a succeed logic and returns the value with the succeedOperation call.

The second part of the service code consists of the main function, which registers the service and the operation just defined, and also publishes the socket_info event.

```
int main() {
    installSigIntHandler();

    Service* s =
    registerService(
        "tcp://robot.franka.de:11511", // master URI
        "tcp://lo", // interface
        "socket-pose"); // service name

    registerOperation(s, // service
        "getSocketPose", // op. name
        GetSocketPoseDescriptor(), // descriptor
        reinterpret_cast<OperationHandler> (
            getSocketPoseCb), // callback
        nullptr); // user data

    Publisher *p = registerEvent(s, // service
        "socket_info", // event name
        SocketInfoDescriptor()); // descriptor

    while (!shutdownSignalled()) { // interrupted?
        spinOnce(s, 1000);
        int number_of_sockets = 0;

        // ...
        // Get number of sockets from the camera
        // ...

        // create message
        SocketInfo socket_info{
            .number_of_sockets = number_of_sockets};
        publish(pub, &socket_info);
    }

    unregisterService(s); // clean up

    return EXIT_SUCCESS;
}
```

The main function of the service is self-explanatory: the registerService function lets the core know about our new service, while the registerOperation does the same with our getSocketPose operation. Thereafter, the service spins and publishes the socket_info event every second until it receives an interruption.

After compiling the service, we can run it as a separate process in our computer. To call it within a state machine, we use the following exemplary code:

```
get_socket_pose {
  port success service("socket-pose")
    .operation("getSocketPose").status ==
    "success"
  port error service("socket-pose")
    .operation("getSocketPose").status ==
    "error"

  resultType
  {
    [16]float socket_pose;
  }

  entry @{
    service ("socket-pose")
    .operation ("getSocketPose").call({})
  }@

  exit @{
    setResult(service("socket-pose")
    .operation("getSocketPose").result)
  }@
}
```

This `get_socket_pose` state machine calls the `getSocketPose` operation of the `socket-pose` service in its entry script. As shown here, state machines can also have `entry` and `exit` scripts, which are executed on entry and right, respectively, after a transition to another state machine is triggered. Similarly, we can access the `socket_info` event with

```
service("socket-pose").event("socket_
info").number_of_sockets
```

You can find the full code in the tutorial repository.

Integration With Other Ecosystems

RIDE integrates any ROS nodes into the system by wrapping their functionality as an external service. Intuitively, ROS topics are similar to RIDE events, while ROS services (or actions with cancellation) correspond to RIDE operations. A tutorial on how to write an ROS/RIDE wrapper can be found in the RIDE documentation.

Programming With the FCI

The FCI, as shown in Figure 9, offers a 1-kHz real-time transparent interface to the robot that allows custom controllers to run on an external computer running Linux with the Preemptive RT patch. It can either fully circumvent Control or rely on the internal

controllers to follow joint or Cartesian space trajectories at 1 kHz, specified with motion interfaces. Specifically, the FCI also provides joint position, joint velocity, Cartesian pose, and Cartesian velocity interfaces. Libfranka, the open source library for the client side of the interface, also provides access to all the sensor measurements and the robot's model library, which includes the kinematic and Lagrangian dynamic model parameters.

To program the plug-in example with the FCI, we implement a Cartesian impedance controller with an additional term that regulates the desired contact force. To do so, we use the joint-torque interface and the model library included in libfranka.

Theoretical Background

Consider the Lagrangian dynamics of the robot:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau_d + \tau_{\text{ext}}, \quad (1)$$

where $M(q)$ is the symmetric, positive-definite mass matrix; $C(q, \dot{q})\dot{q}$ is the Coriolis and centrifugal torques; $g(q)$ is the gravity vector; τ_{ext} is the vector of external link-side joint torques; and τ_d is the control input.

To wiggle orthogonal to the applying force, we implement a Cartesian impedance controller for executing wiggling in the (x, y) -plane, while at the same time a naive proportional integral (PI) force controller will control the external force in the z -axis as follows:

$$\tau_d = \tau_{\text{cart}} + \tau_{\text{force}} + C(q, \dot{q})\dot{q}, \quad (2)$$

where

$$\tau_{\text{cart}} = J^T(q)(Ke + D\dot{e}) \quad (3)$$

$$\tau_{\text{force}} = J^T(q)(f_d + K_p e_f + K_i \int e_f dt) \quad (4)$$

where f_d is the desired wrench; $x_d = \{p_d, q_d\}$ is the desired pose; e and e_f are the pose and force error, respectively; K , D are the Cartesian stiffness and damping, respectively; and K_i is the integral term gain of the PI force control (see [16] for more details). Note that (2) does not include the gravity term $g(q)$ from (1) as the FCI interface already compensates for it.

For wiggling, the orientation of the equilibrium pose is rotated around the x - and y -axes of the initial orientation with angles

$$\begin{aligned} a_x(t) &= \alpha_x \sin(2\pi\omega_x t) \\ a_y(t) &= \alpha_y \cos(2\pi\omega_y t). \end{aligned} \quad (5)$$

Implementation

For the sake of clarity, we show the most relevant excerpts of the code, implemented in C++. First, we

establish a connection with the robot and read the initial state with the `readOnce` function of the `franka::Robot` object:

```
// connect to robot
franka::Robot robot(robot_ip);
// load the kinematics and dynamics model
franka::Model model = robot.loadModel();

franka::RobotState initial_state =
    robot.readOnce();

Eigen::Map<Eigen::Matrix<double, 6, 1>>
    force_ext_initial(initial_state.O_F_
        ext_hat_K.data());
force_error_integral.setZero();

// equilibrium point is the initial position
Eigen::Affine3d initial_transform(Eigen::
    Matrix4d::Map(initial_state.O_T_EE.
        data()));
Eigen::Vector3d position_d(initial_
    transform.translation());
Eigen::Quaterniond orientation_initial
    (initial_transform.linear());

double time = 0.0;
```

Note that to also correct the slightest bias that might be present in the external force estimation, we store the initial force (before establishing contact) as a bias correction, and we initialize the integral term in (4) to 0. The force estimation during motion (especially during high accelerations) is less accurate in comparison to the quasi-static case known from previous literature. This is an inherent property of the implemented disturbance observer. The static bias can further be caused by imprecise vertical mounting of the robot base, imprecise compensation of the end effector's mass and inertia tensor, or a missing update of finger dynamics.

A callback function for our controller is defined as a C++11 function as

```
// define callback for the torque control
loop
std::function<franka::Torques(
    const franka::RobotState&, franka::
    Duration)>
plug_in_control_callback =
    [&] (const franka::RobotState& robot_state,
        franka::Duration period) ->
        franka::Torques {
    // ... code for callback
}
```

This function, which receives the robot state and duration of the cycle, will be called every millisecond until a

`franka::MotionFinished` signal is sent by the callback. We can then have a look at the callback code that implements (5) and (2). We first cast all the involved variables to [28] to easily perform matrix products and quaternion manipulation.

```
// update time
time += period.toSec();

// get state variables
std::array<double, 7> coriolis_array =
    model.Coriolis(robot_state);
std::array<double, 42> jacobian_array =
    model.zeroJacobian(franka::Frame::kEnd
        Effector, robot_state);

// convert to Eigen
Eigen::Map<const Eigen::Matrix<double,
    7, 1>> coriolis(coriolis_array.data());
Eigen::Map<const Eigen::Matrix<double,
    6, 7>> Jacobian(jacobian_array.data());
Eigen::Map<const Eigen::Matrix<double,
    6, 1>> force_ext(robot_state.O_F_ext
        _hat_K.data());
Eigen::Map<const Eigen::Matrix<double,
    7, 1>> dq(robot_state.dq.data());
Eigen::Affine3d transform(Eigen::Matrix4d
    ::Map(robot_state.O_T_EE.data()));
Eigen::Vector3d position(transform.
    translation());
Eigen::Quaterniond orientation(transform.
    linear());
```

To implement (5), the orientation of the equilibrium pose is computed as

```
Eigen::AngleAxisd angle_axis_wiggle_x;
angle_axis_wiggle_x.axis() << 1, 0, 0;
angle_axis_wiggle_x.angle() = sin(2.0 *
    M_PI * time * plug_in_params.wiggle_
    frequency_x) * plug_in_params.wiggle_
    amplitude_x;
Eigen::AngleAxisd angle_axis_wiggle_y;
angle_axis_wiggle_y.axis() << 0, 1, 0;
angle_axis_wiggle_y.angle() = sin(2.0 *
    M_PI * time * plug_in_params.wiggle_
    frequency_y) * plug_in_params.wiggle_
    amplitude_y;

Eigen::Quaterniond wiggle_x(angle_axis
    _wiggle_x);
Eigen::Quaterniond wiggle_y(angle_axis
    _wiggle_y);
Eigen::Quaterniond orientation_d
    (wiggle_y * (
        wiggle_x * orientation_initial));
```

The error signals e and e_f that drive our controller are given by

```
// Compute error to desired equilibrium pose
// position error
Eigen::Matrix<double, 6, 1> error;
error.head(3) << position - position_d;

// orientation error
if (orientation_d.coeffs().dot(orientation.
    coeffs()) < 0.0) {
    orientation.coeffs() << -orientation.
        coeffs();
}
// "difference" quaternion
Eigen::Quaterniond error_quaternion
    (orientation.inverse() * orientation_d);
error.tail(3) << error_quaternion.x(),
    error_quaternion.y(), error_
        quaternion.z();
// Transform to base frame
error.tail(3) << -transform.linear() *
    error.tail(3);

// Compute error to desired force removing
    initial bias
force_error = desired_force - force + force
    _initial;
force_error_integral = force_error_integral +
    period.toSec() * force_error;
```

We can now easily compute the force controller (4), Cartesian impedance controller (3), and desired torque (2) as

```
// Compute control
Eigen::VectorXd force_control(6), tau_
    force(7), tau_cart(7), tau_cmd(7);

// Force control term
force_control = desired_force + k_p *
    force_error + k_i * force_error_
        integral;
force_control << 0, 0, force_control(2),
    0, 0, 0;
tau_force = jacobian.transpose() *
    force_control;

// Cartesian control term
tau_cart << jacobian.transpose() *
    (-cartesian_stiffness * error -
        cartesian_damping * (jacobian *
            dq));

// Commanded torque
tau_cmd << tau_cart + tau_force + coriolis;
```

Finally, to start the controller, we simply call the control function.

```
robot.control(plugin_control_callback);
```

Although more intricate, the FCI provides full control over the system; it was designed for researchers in control theory, signal processing, and low-level dynamic planning.

Integration With Other Ecosystems

Other relevant frameworks are also compatible with the FCI, including support for

- **MATLAB Simulink.** You will receive full access to all the real-time interfaces and dynamic model parameters in our Simulink Toolbox.
- **ROS,** including integration into the MoveIt! planning framework, from which the Franka Emika robot is the standard tutorial robot. We provide additional support for multiarm control (from version 0.7.0, this feature was officially added with examples), external grippers, and vision.
- **The NVIDIA Isaac SDK,** which aims for accelerating robotic AI-based solutions, with a strong emphasis on simulation and learning.

Robot Learning With the Franka Emika Robot

Manipulation skill learning is one of the most attractive approaches for researchers and industry as it enables cost-effective automation of otherwise too-challenging or expensive use cases. A generic and intuitive description of this problem consists of finding the control parameters that succeed at performing a task and possibly also minimize a specific cost, e.g., energy consumption. For the plug-in example, we implement a procedure that finds the parameters for the state machine from the “Programming With RIDE” section that succeed at inserting the plug and, at the same time, minimize the interaction forces. Regardless of the method used, e.g., reinforcement learning [29], [30] or stochastic optimization [31], all on-robot learning methods follow the same underlying operational loop, where a learning service generates parameters that are then tested and evaluated on the robot. The result is used by the learning service to generate the next parameter set, and this procedure is iterated until convergence. In our prototypical implementation, the learning service consists of the following three operations: `init`, `getParams`, and `setResult`. To implement the learning loop, the `plug_in_learning` state machine first initializes the algorithm in the `init` state by calling the `init` operation of the learning service. This operation starts the algorithm by setting hyperparameters and initializing memory.

```
plug_in_learning_ride {
    port success child("set_result")
        .port("learning_done")
    port error child("init").port("error") or
        child("get_params").port
            ("error") or
        -- ...
```

```
--> init {
  port success ... -> reset
  port error ...

  -- Define learning hyperparameter,
  e.g.:
  parameterType {
    {} hyperparameters;
  }

  entry @{
    service("learning")
      .operation("init")
      .call(parameter.hyperparameters)
  @}
}
```

First, to prepare the robot for the next iteration, the reset state resets the robot to an initial state and recovers from any errors. With this state, the learning loop starts.

```
reset {
  port success child("home")
    .port("success") -> get_params
  port error ...

  --> reset_errors <- reset_errors_
  library {
    port success -> home
  }

  home <- move_via_with_move_configuration {
    port success -> error_recovery
  } where {
    -- set move via parameters
  }
}
```

Then, the get_params state requests another operation for getting the parameters for the next experimental trial.

```
get_params {
  port success ... -> explore
  port error ...

  -- define all parameter being optimized,
  e.g.,:
  resultType {
    {} plug_in_params;
  }

  entry @{
    service("learning")
      .operation("getParams").call({})
  @}
}
```

```
exit @{
  local res = service("learning")
    .operation("getParams").
    result
  -- set plug_in_params
  setResult({plug_in_params = {...}})
  @}
}
```

Using the resulting parameters, the plug_in state machine from the “Programming With the Franka Emika Robot” section is executed, and at the same time, the robot state is observed to compute the cost value.

```
explore {
  port done ... -> set_result

  -- define all parameter for plug_in
  parameterType {
    {} plug_in_params;
    -- ...
  }

  --> barrier plug_in_and_observe {
    -> plug_in
    -> observe
  }

  -- Run the plug_in controller from the
  "Programming With RIDE" section
  plug_in <- plug_in {
  } where parameter

  observe {
    resultType {
      float cost;
    }

    action service("robot")
      .operation("robot_state") ~=
      nil @{
        -- compute cost function
      @}
    }
  } where {
    plug_in_params: child("get_params")
      .result.plug_in_
      params;
    -- ...
  }
```

Regardless of the plug_in state machine execution outcome, the set_result state returns the result of the cost function and a Boolean representing the insertion success by calling the setResult operation, which makes the result available to the learning algorithm.

```

set_result {
  port learning_done ...
  port continue ... -> reset
  port error ...
  parameterType {
    float cost;
    bool is_inserted;
  }

  entry @{
    service("learning")
      .operation("setResult") .
      call(parameter)
  }
  @}
} where child("explore").result;

```

The learning loop is repeated until the port `learning_done` in the `set_result` state is triggered, which is determined by the learning algorithm running in the service. A full implementation of this example can be found in the tutorial repository.

Learning With a Custom Controller

Although the previous section learns the optimal parameters for the plug-in programmed with RIDE, we can use the same code with small modifications to optimize the parameters of a custom FCI controller, such as the one from the “Programming With the FCI” section. The only difference lies on the execution of the plug-in state machine, where, instead, we call the operation *plugIn* of an additional *fc*i service.

```

plug_in {
  port success ...
  port error ...

  -- define parameter for the plug in
  controller
  parameterType {
    {} plug_in_params;
  }
  -- ...
}

resultType {
  string error_cause;
}

entry @{
  service("fc")
    .operation("plugIn") .call(parameter)
  @}

exit @{
  setResult(service("fc")
    .operation("plugIn") .
    result)
}

```

```

@}
} where {
  plug_in_params: parameter.plug_in_
    params;
  -- ...
}

```

The following code snippet illustrates the implementation of the *fc*i service and its *plugIn* operation. This operation runs the plug-in *fc*i controller as described in the “Programming With the FCI” section: it handles the robot connection, loads the models, and starts the controller. All of this is encapsulated into the following *plugIn* callback function:

```

#include <racecom/racecom.h>
#include "messages/PlugIn.h"

#include <franka/robot.h>

void plugIn(CallHandle h, // call
  handle
  struct PlugIn_request* request,
  void* hint) { // user
  data
  PlugIn_result result;
  // ...
  // Set parameter of plug-in controller
  // ...
  // and run plug-in FCI controller
  // as implemented in the "Programming
  With the FCI" section
  franka::Robot robot(robot_ip);
  // ...
  robot.control(plug_in_control_callback);
  // ...
  succeedOperation(h, &result); // send
  result
  free_PlugIn_request(request);
}

```

As shown in Figure 10, this example connects the FCI to the core through the FSI. Note that any concurrent commands from the robot service and the FCI will be rejected. A full implementation of this example can be found at <https://github.com/frankaemika>.

Franka Community

At the 2018 IEEE International Conference on Robotics and Automation (ICRA), we organized “A Hands-On Tutorial on the Franka Emika Robot,” the basis for this article. There we presented how a task can be solved using the different interfaces of the robot and invited the audience to test them out and experience the different approaches firsthand. In the meantime, the platform has been made available worldwide and has already been used in a wide variety of fields, including control theory, teleoperation [32], [33], computer vision

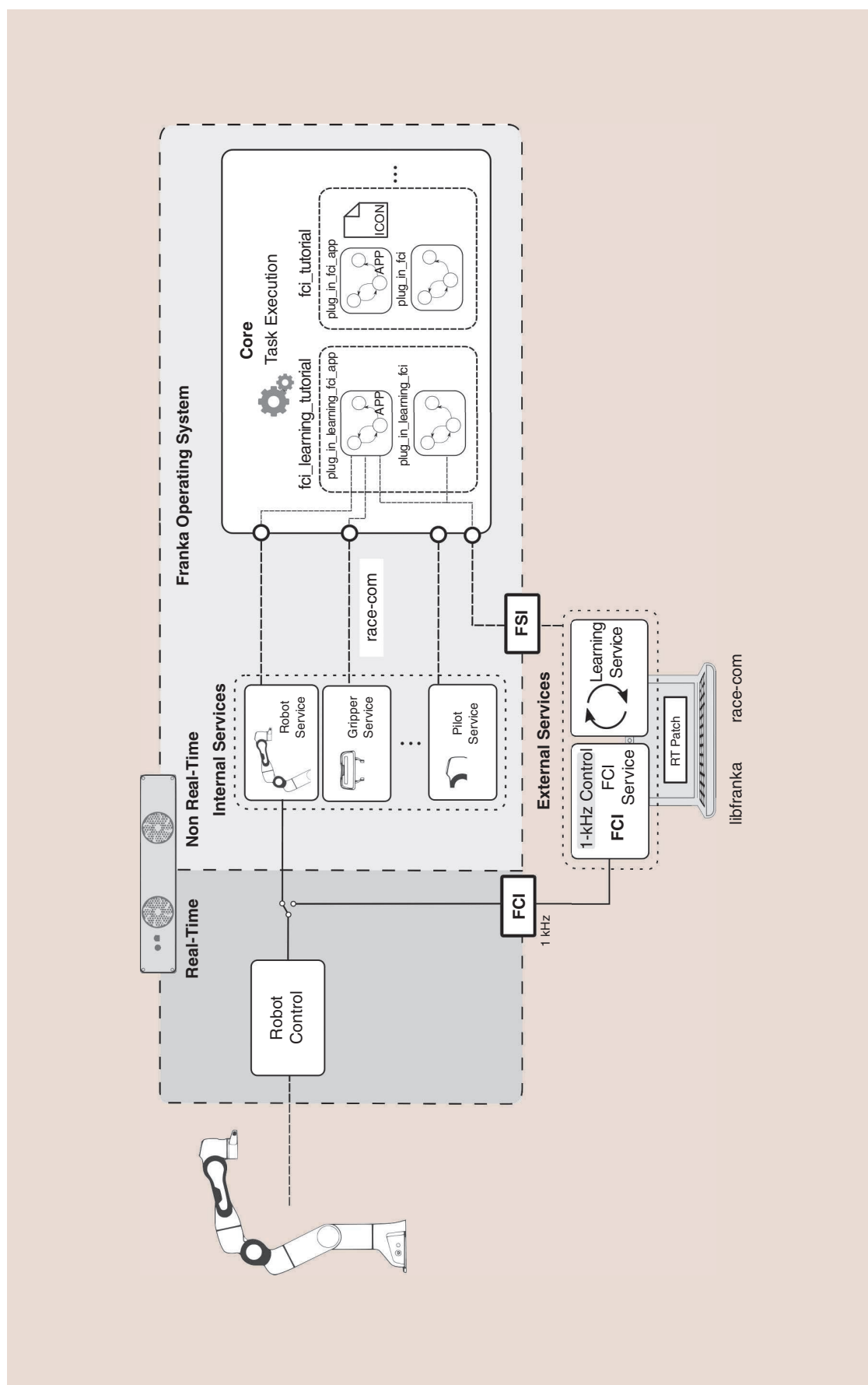


Figure 10. An overview of the hardware and software architecture for the learning example with a custom FCI controller.

[34], human–robot interaction [35], [36], health care [37], ML [31], [38]–[40], and robot modeling and identification [41] as well as education [42]. The ICRA 2018 workshop was also the scenario where we launched Franka Community [43], an online platform to support this already-significant user base. In this forum, the Franka Emika Robot users—and robotics enthusiasts in general—can access Franka Community, an online forum used to foster discussions, find help, share codes and results, or look for potential collaborations.

Conclusions

This article is a tutorial on the Franka Emika robot, the first series of industrial, AI-ready tactile robot platforms designed for the full spectrum of robotic research and education domains. It provides three different interfaces: Desk, for high level App-based programming; RIDE, for customizing apps and integrating sensors and services; and the FCI, for writing custom, low-level controllers. We presented exemplary code on how to use each interface, apply manipulation learning, and combine all interfaces. This flexible platform is also supported by a lively and already-substantial community. In the future, we aim for making comparisons and benchmarks easier for researchers to accelerate the development and objective advancement of the state of the art in robot manipulation.

Acknowledgments

This work was financially supported by the Bavarian Ministry for Economic Affairs, Regional Development and Energy (StMWi) as part of the projects “iSTRAP,” “Project X” (IUK-1807-0007// IUK582/001), “KI.FABRIK” (DIK-2103-005//DIK0249/01) and “SAM: Safe- Autonomous-Mobile-Robot” (IUK-1812-0005//IUK- 6119/004), and the German Research Foundation (DFG, Deutsche Forschungsgemeinschaft) as part of Germany’s Excellence Strategy EXC 2050/1 Project ID 390696704 Cluster of Excellence Centre for Tactile Internet with Human-in-the-Loop (CeTI) of Technische Universität Dresden. We further gratefully acknowledge the funding of project KoBo34 (Verbundprojekt Nummer V5ARA202) by the BMBF (grant 16SV7985). This work was also supported by the European Union’s Horizon 2020 research and innovation programme as part of the project I.AM. under grant 871899. We would like to thank Samuel Schneider and Peter So for their valuable comments and feedback to improve the article and its material.

References

- [1] “Data sheet 1 robot arm and control,” Franka Emika GmbH, 2020. https://download.franka.de/Franka_Emika_Panda_Data_sheet_EN-2020_04.pdf
- [2] Universal Robots “UR/URe series,” 2008. [Online]. Available: <https://robots.ieee.org/robots/universal/>
- [3] ABB, “Yumi,” 2011. [Online]. Available: <https://robots.ieee.org/robots/yumi/>
- [4] “Rethink robotics sawyer,” Robots, 2015. <https://robots.ieee.org/robots/sawyer/>
- [5] “Barret WAM,” Robots, 2004. <https://robots.ieee.org/robots/wam/>
- [6] “LBR iiwa,” Robots, 2013. <https://robots.ieee.org/robots/lbriiwa/>
- [7] W. Krefl, Inverse kinematics determination and trajectory tracking algorithm development of a robot with 7 joints,” in *Proc. IEEE 2020 16th Int. Conf. Control, Autom., Robot., and Vision (ICARCV)*, pp. 1001–1007.
- [8] S. Haddadin, S. Schneider, and L. Johannsmeier, *Franka Video Tutorial* (2021). [Online Video]. Available: <https://www.youtube.com/playlist?list=PLeVqiD9ILCrLJI3Eibi-je2fNqwt1am>
- [9] “Franka Emika GmbH: Tutorial code,” GitHub, 2021. https://github.com/frankaemika/the_reference_platform
- [10] G. Hirzinger, A. Albu-Schäffer, M. Hähne, I. Schäffer, and N. Sporer, “On a new generation of torque controlled light-weight robots,” in *Proc. 2001 ICRA. IEEE Int. Conf. Robot. Automat. (Cat. No. 01CH37164)*, vol. 4, pp. 3356–3363, doi: 10.1109/ROBOT.2001.933136.
- [11] G. Hirzinger *et al.*, “DLR’s torque-controlled light weight robot III—are we reaching the technological limits now?” in *Proc. 2002 IEEE Int. Conf. Robot. Automat. (Cat. No. 02CH37292)*, vol. 2, pp. 1710–1716, doi: 10.1109/ROBOT.2002.1014788.
- [12] A. Albu-Schäffer, S. Haddadin, C. Ott, A. Stemmer, T. Wimböck, and G. Hirzinger, “The DLR lightweight robot – Design and control concepts for robots in human environments,” *Ind. Robot, Int. J.*, vol. 34, no. 5, pp. 376–385, 2007, doi: 10.1108/01439910710774386.
- [13] B. Rooks, “The harmonious robot,” *Ind. Robot, Int. J.*, vol. 33, no. 2, pp. 125–130, 2006, doi: 10.1108/01439910610651446.
- [14] C. Fitzgerald, “Developing baxter,” in *Proc. 2013 IEEE Conf. Technol. Practical Robot Appl. (TePRA)*, pp. 1–6, doi: 10.1109/TePRA.2013.6556344.
- [15] N. Hogan, “Impedance control: An approach to manipulation,” in *Proc. 1984 Amer. Contr. Conf.*, pp. 304–313, doi: 10.23919/ACC.1984.4788393.
- [16] O. Khatib, “A unified approach for motion and force control of robot manipulators: The operational space formulation,” *IEEE J. Robot. Automat.*, vol. 3, no. 1, pp. 43–53, 1987, doi: 10.1109/JRA.1987.1087068.
- [17] A. Albu-Schäffer, C. Ott, U. Frese, and G. Hirzinger, “Cartesian impedance control of redundant robots: Recent results with the DLR-light-weight-arms,” in *Proc. 2003 IEEE Int. Conf. Robot. Automat. (Cat. No. 03CH37422)*, vol. 3, pp. 3704–3709, doi: 10.1109/ROBOT.2003.1242165.
- [18] C. Ott, A. Albu-Schäffer, A. Kugi, S. Stamigioli, and G. Hirzinger, “A passivity based cartesian impedance controller for flexible joint robots - Part I: Torque feedback and gravity compensation,” in *Proc. 2004 IEEE Int. Conf. Robot. Automat., ICRA’04*, vol. 3, pp. 2659–2665, doi: 10.1109/ROBOT.2004.1307462.
- [19] A. Albu-Schäffer, C. Ott, and G. Hirzinger, “A passivity based cartesian impedance controller for flexible joint robots - Part II: Full state feedback, impedance design and experiments,” in *Proc. 2004 IEEE Int. Conf. Robot. Automat., ICRA’04*, vol. 3, pp. 2666–2672, doi: 10.1109/ROBOT.2004.1307463.
- [20] A. Albu-Schäffer, C. Ott, and G. Hirzinger, “A unified passivity-based control framework for position, torque and impedance control of flexible joint robots,” *Int. J. Robot. Res.*, vol. 26, no. 1, pp. 23–39, 2007, doi: 10.1177/0278364907073776.
- [21] A. De Luca, A. Albu-Schäffer, S. Haddadin, and G. Hirzinger, “Collision detection and safe reaction with the DLR-III lightweight manipulator arm,” in *Proc. 2006 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, pp. 1623–1630, doi: 10.1109/IROS.2006.282053.

- [22] S. Haddadin, L. Johannsmeier, and F. D. Ledezma, "Tactile robots as a central embodiment of the tactile internet," *Proc. IEEE*, vol. 107, no. 2, pp. 471–487, 2018, doi: 10.1109/JPROC.2018.2879870.
- [23] S. Haddadin, A. Albu-Schäffer, and G. Hirzinger, "Requirements for safe robots: Measurements, analysis and new insights," *Int. J. Robot. Res.*, vol. 28, nos. 11–12, pp. 1507–1527, 2009, doi: 10.1177/0278364909343970.
- [24] S. Haddadin, A. Albu-Schäffer, M. Frommberger, J. Rossmann, and G. Hirzinger, "The 'DLR crash report': Towards a standard crash-testing protocol for robot safety - Part I: Results," in *Proc. 2009 IEEE Int. Conf. Robot. Automat.*, pp. 272–279, doi: 10.1109/ROBOT.2009.5152602.
- [25] S. Haddadin, A. De Luca, and A. Albu-Schäffer, "Robot collisions: A survey on detection, isolation, and identification," *IEEE Trans. Robot.*, vol. 33, no. 6, pp. 1292–1312, 2017, doi: 10.1109/TRO.2017.2723903.
- [26] J. Bohren *et al.*, "Towards autonomous robotic butlers: Lessons learned with the PR2," in *Proc. 2011 IEEE Int. Conf. Robot. Automat.*, pp. 5568–5575, doi: 10.1109/ICRA.2011.5980058.
- [27] "Prototyping package," Franka Emika GmbH, 2021. <https://world.franka.de/19886655/ai-robotics-package>
- [28] G. Guennebaud *et al.*, "Eigen is a C++ template library for linear algebra: Matrices, vectors, numerical solvers, and related algorithms," Eigen v3, 2010. <http://eigen.tuxfamily.org>
- [29] J. Kober and J. Peters, "Learning motor primitives for robotics," in *Proc. 2009 IEEE Int. Conf. Robot. Automat.*, pp. 2112–2118, doi: 10.1109/ROBOT.2009.5152577.
- [30] J. Kober and J. R. Peters, "Policy search for motor primitives in robotics," in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 849–856.
- [31] L. Johannsmeier, M. Gerchow, and S. Haddadin, "A framework for robot manipulation: Skill formalism, meta learning and adaptive control," in *Proc. 2019 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 5844–5850, doi: 10.1109/ICRA.2019.8793542.
- [32] M. Laghi *et al.*, "Shared-autonomy control for intuitive bimanual tele-manipulation," in *Proc. 2018 IEEE-RAS 18th Int. Conf. Humanoid Robots (Humanoids)*, pp. 1–9, doi: 10.1109/HUMANOIDS.2018.8625047.
- [33] B. Omarali, F. Palermo, M. Valle, S. Poslad, K. Althoefer, and I. Farkhatdinov, "Position and velocity control for telemanipulation with interoperability protocol," in *Proc. 20th Towards Auton. Robot. Syst. Conf., Lecture Notes Artif. Intell.*, London, Jul. 2019, pp. 316–324, doi: 10.1007/978-3-030-23807-0_26.
- [34] A. Roberti, R. Muradore, P. Fiorini, M. Cristani, and F. Setti, "An energy saving approach to active object recognition and localization," in *Proc. IECON - 44th Annu. Conf. IEEE Ind. Electron. Soc.*, Oct. 2018, pp. 3153–3158, doi: 10.1109/IECON.2018.8591411.
- [35] M. Jorda, E. G. Herrero, and O. Khatib, "Contact-driven posture behavior for safe and interactive robot operation," in *Proc. Int. Conf. Robot. Automat. (ICRA)*, May 2019, pp. 9243–9249, doi: 10.1109/ICRA.2019.8793691.
- [36] P. Balatti, D. Kanoulas, N. Tsagarakis, and A. Ajoudani, "Towards robot interaction autonomy: Explore, identify, and interact," in *Proc. Int. Conf. Robot. Automat. (ICRA)*, Apr. 2019, pp. 9523–9529, doi: 10.1109/ICRA.2019.8794428.
- [37] J. Grischke, L. Johannsmeier, L. Eich, and S. Haddadin, "Dentronics: Review, first concepts and pilot study of a new application domain for collaborative robots in dental assistance," in *Proc. Int. Conf. Robot. Automat. (ICRA)*, May 2019, pp. 6525–6532, doi: 10.1109/ICRA.2019.8794139.
- [38] Y. Chebotar *et al.*, "Closing the sim-to-real loop: Adapting simulation randomization with real world experience," 2018, arXiv:1810.05687.
- [39] L. Berscheid, P. Meißner, and T. Kröger, "Robot learning of shifting objects for grasping in cluttered environments," 2019, arXiv:1907.11035.
- [40] S. Nair and C. Finn, "Hierarchical foresight: Self-supervised learning of long-horizon tasks via visual subgoal generation," Sep. 2019, arXiv:1909.05829.
- [41] C. Gaz, M. Cognetti, A. Oliva, P. Robuffo Giordano, and A. De Luca, "Dynamic identification of the Franka Emika robot with retrieval of feasible parameters using penalty-based optimization," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 4147–4154, Oct. 2019, doi: 10.1109/LRA.2019.2931248.
- [42] S. Haddadin *et al.*, "Roboterfabrik: A pilot to link and unify German robotics education to match industrial and societal demands," in *Proc. 2018 Companion ACM/IEEE Int. Conf. Human-Robot Interact.*, ACM, pp. 375–375, doi: 10.1007/978-3-319-97085-1_1.
- [43] S. Haddadin and S. Parusel, "Franka community," Franka Emika, 2018. <https://www.franka-community.de/>

Sami Haddadin, Munich Institute of Robotics and Machine Intelligence and Chair of Robotics and Systems Intelligence, Technical University of Munich, Munich, 80992, Germany. Email: haddadin@tum.de.

Sven Parusel, Franka Emika GmbH, Munich, 80797, Germany. Email: info@franka.de.

Lars Johannsmeier, Munich Institute of Robotics and Machine Intelligence and Chair of Robotics and Systems Intelligence, Technical University Munich, Munich, 80992 Germany. Email: lars.johannsmeier@tum.de.

Saskia Golz, Franka Emika GmbH, Munich, 80797, Germany. Email: info@franka.de.

Simon Gabl, Franka Emika GmbH, Munich, 80797, Germany. Email: info@franka.de.

Florian Walch, Franka Emika GmbH, Munich, 80797, Germany. Email: info@franka.de.

Mohamadreza Sabaghian, Franka Emika GmbH, Munich, 80797, Germany. Email: info@franka.de.

Christoph Jähne, Franka Emika GmbH, Munich, 80797, Germany. Email: info@franka.de.

Lukas Hausperger, Franka Emika GmbH, Munich, Germany. Email: info@franka.de.

Simon Haddadin, Franka Emika GmbH, Munich, 80797, Germany. Email: info@franka.de.